МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Объектно-ориентированное программирование»
Тема: Полиморфизм

Студент гр. 3344 _____ Мурдасов М.К.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2024

Цель работы

Создать класс-интерфейс способности. Создать класс менеджер способностей, который хранит очередь способностей. Реализовать функционал получения случайной способности при уничтожении вражеского корабля. Реализовать набор классов-исключений.

Задание

Лабораторная работа №2 - Полиморфизм

a) Создать класс-интерфейс способности, которую игрок может применять. Через наследование создать 3 разные способности:

i) Двойной урон - следующая атак при попадании по кораблю нанесет сразу 2 урона (уничтожит сегмент).

ii) Сканер - позволяет проверить участок поля 2x2 клетки и узнать, есть ли там сегмент корабля. Клетки не меняют свой статус.

iii) Обстрел - наносит 1 урон случайному сегменту случайного корабля. Клетки не меняют свой статус.

b) Создать класс менеджер-способностей. Который хранит очередь способностей, изначально игроку доступно по 1 способности в случайном порядке. Реализовать метод применения способности.

c) Реализовать функционал получения одной случайной способности при уничтожении вражеского корабля.

d) Реализуйте набор классов-исключений и их обработку для следующих ситуаций (можно добавить собственные):

i) Попытка применить способность, когда их нет

ii) Размещение корабля вплотную или на пересечении с другим кораблем

iii) Атака за границы поля

Примечания:

- Интерфейс события должен быть унифицирован, чтобы их можно было единообразно использовать через интерфейс
- Не должно быть явных проверок на тип данных

Выполнение работы

Класс IAbility

Описание:

Абстрактный класс *IAbility* является интерфейсом способности. Он состоит только из чистых виртуальных public методов, не имеющих в нем определения и не содержит никаких полей-данных. Предназначен для хранения общей реализации и поведения способностей.

Методы класса:

- **Метод активации способности:**
    - *virtual void apply(ResultOfUsingAbilities& results) = 0;* - активирует способность.
- **Деструктор:**
    - *virtual ~AbilityInterface() = default;* - деструктор по умолчанию.

Таким образом, класс *IAbility* предоставляет общую реализацию всех способностей. Сами способности в дальнейшем будут наследоваться от него.

Класс DoubleDamage

Описание:

Класс *DoubleDamage* является реализацией интерфейса *IAbility* и представляет собой способность наносящую двойной урон по клетке.

Методы класса:

- **Метод активации способности:**
    - *void apply(ResultOfUsingAbilities& results);* - устанавливает статус способности в Activated для использования при следующей атаке.

Класс RandomHit

Описание:

Класс *RandomHit* является реализацией интерфейса *IAbility* и представляет собой способность наносящую урон случайному сегменту случайного корабля на поле.

Поля класса:

- *GameField& field;* - хранит ссылку на поле.

Методы класса:

- **Конструктор:**
    - *RandomHit(GameField& field);* - создает объект способности.
- **Метод активации способности:**
    - *void apply(ResultOfUsingAbilities& results);* - генерирует случайный индекс корабля, затем вызывает у выбранного корабля метод hitSegment() по случайному индексу и передает результат работы в *ResultOfUsingAbility*.

Класс Scanner

Описание:

Класс *Scanner* является реализацией интерфейса *IAbility* и представляет собой способность просканировать область размером 2x2 на поле и получить информацию о нахождении в ней корабля.

Поля класса:

GameField& field; - хранит ссылку на поле.

int x, y; - хранят значения координат левой верхней точки области.

Методы класса:

- **Конструктор:**
    - *Scanner(GameField& field, int x, int y);* - создает объект способности.
- **Метод активации способности:**
    - *void apply(ResultOfUsingAbilities& results);* - проверяет статус клеток в выбранной области, сообщает, если какой-то из них

оказался *CellState::Ship* и передает результат работы в *ResultOfUsingAbility*.

- **Методы проверки параметров:**
    - *bool checkScanCoords(int y, int x);* - проверяет область сканирования на корректность.

Класс AbilityManager

Описание:

Класс *AbilityManager* отвечает за использование доступных способностей, выдачу способностей пользователю и добавление их в очередь.

Поля класса:

GameField& field; - хранит ссылку на поле.

std::queue<IFactory> abilityQueue* – очередь способностей.

Методы класса:

- **Конструктор:**
    - *AbilityManager(GameField& field);* - инициализирует менеджер и вызывает *getStarterAbility().*
- **Методы управления данными:**
    - *void addRandomAbility();* - добавляет в очередь случайную способность.
    - *void useAbility(ResultOfUsingAbilities& results);* - активация первой в очереди способности.
    - *void getStarterAbility();* - инициализирует первые 3 способности в случайном порядке и добавляет их в очередь.
    - *std::string availableAbilities();* - возвращает информацию об очереди доступных способностей.

Класс ResultOfUsingAbilities

Описание:

Класс *ResultOfUsingAbilities* хранит результаты применения способностей.

Поля класса:

- *LastAbilityUsed lastAbilityUsed;* - хранит информацию о последней использованной способности.
- *DoubleDamageResult DDResult;* - хранит информацию об активации способности Double Damage.
- *ScannerResult ScanResult;* - хранит информацию о результате работы способности Scanner.

Методы класса:

- *void setDoubleDamageResult(DoubleDamageResult result);* - устанавливает информацию об активации способности Double Damage.
- *void setScannerResult(ScannerResult result);* - устанавливает информацию о результатах работы способности Scanner.
- *void setLastAbilityUsed(LastAbilityUsed last);* - устанавливает информацию о последней использованной способности.
- *DoubleDamageResult getDoubleDamageResult() const;* - возвращает информацию об активации способности Double Damage.
- *ScannerResult getScannerResult() const;* - возвращает информацию о результатах работы способности Scanner.
- *LastAbilityUsed ScanResult;* - возвращает информацию о последней использованной способности.
- *void printResultOfUsing();* - выводит в консоль сопровождающую информацию об использовании способностей.

Структуры:

- *enum class DoubleDamageResult {ACTIVATED, DEACTIVATED};* - статус активации способности DoubleDamage.
- *enum class ScannerResult{Found, NotFound, NoInformation};* - результат работы способности Scanner.

- *enum class LastAbilityUsed{DoubleDamage, Scanner, RandomHit};* - последняя использованная способность.

Класс IFactory

Описание:

Класс *IFactory* представляет собой «фабрику» для создания и выдачи способностей.

Методы класса:

- **Метод создания способности:**
  - *virtual IAbility* createAbility() = 0;* - возвращает указатель на созданную способность.
- **Информационные методы:**
  - *virtual std::string getInfo() const = 0;* - возвращает название создаваемой способности.
- **Деструктор:**
  - *virtual ~IFactory() = default;* - деструктор по умолчанию.

Класс DoubleDamageFactory

Описание:

Класс *DoubleDamageFactory* является реализацией интерфейса *IFactory.* Создает объект способности *DoubleDamage* и возвращает указатель на нее.

Методы класса:

- **Метод создания способности:**
  - *IAbility* createAbility();* - возвращает указатель на созданную способность *DoubleDamage*.
- **Информационные методы:**
  - *std::string getInfo() const*; - возвращает *"[Double Damage]"*.

Класс RandomHitFactory

Описание:

Класс *RandomHitFactory* является реализацией интерфейса *IFactory*. Создает объект способности *RandomHit* и возвращает указатель на нее.

Поля класса:

- *GameField& field;* - ссылка на поле.

Методы класса:

- **Конструктор:**
  - *RandomHitFactory(GameField& field);*
- **Метод создания способности:**
  - *IAbility* createAbility();* - возвращает указатель на созданную способность *RandomHit*.
- **Информационные методы:**
  - *std::string getInfo() const;* - возвращает *"[Random Hit]"*.

Класс ScannerFactory

Описание:

Класс *ScannerFactory* является реализацией интерфейса *IFactory.* Создает объект способности *Scanner* и возвращает указатель на нее.

Поля класса:

- *GameField& field;* - ссылка на поле.
- *CoordsHolder coordsHolder;* - обеспечивает получение координат.

Методы класса:

- **Конструктор:**
  - *ScannerFactory(GameField& field);*
- **Метод создания способности:**
  - *IAbility* createAbility();* - возвращает указатель на созданную способность *Scanner*.
- **Информационные методы:**
  - *std::string getInfo() const;* - возвращает *"[Scanner]"*.

Класс InfoHolder

Описание:

Интерфейс *InfoHolder* определяет поведение для структур хранения данных.

Методы класса:

- *virtual void need(std::string name) = 0; - запрашивает параметр у считывателя.*
- *virtual std::variant<int, std::string, bool> get(std::string name) = 0;* - возвращает параметр по имени. Расчитан на работу с любыми холдерами.
- *~InfoHolder() = default;* - деструктор по умолчанию.

Класс CoordsHolder

Описание:

Класс *CoordsHolder* запрашивает и хранит координаты. Является реализацией интерфейса *InfoHolder*.

Поля класса:

- *CoordsReader reader;* - отвечает за считывание данных из консоли.
- *int x, y – координаты.*

Методы класса:

- *void need(std::string name);* - запрашивает у ридера координату.
- *std::variant<int, std::string, bool> get(std::string name);* - возвращает координату по имени.

Класс CoordsReader

Описание:

Класс *CoordsReader* считывает координату из консоли. Используется холдером.

Методы класса:

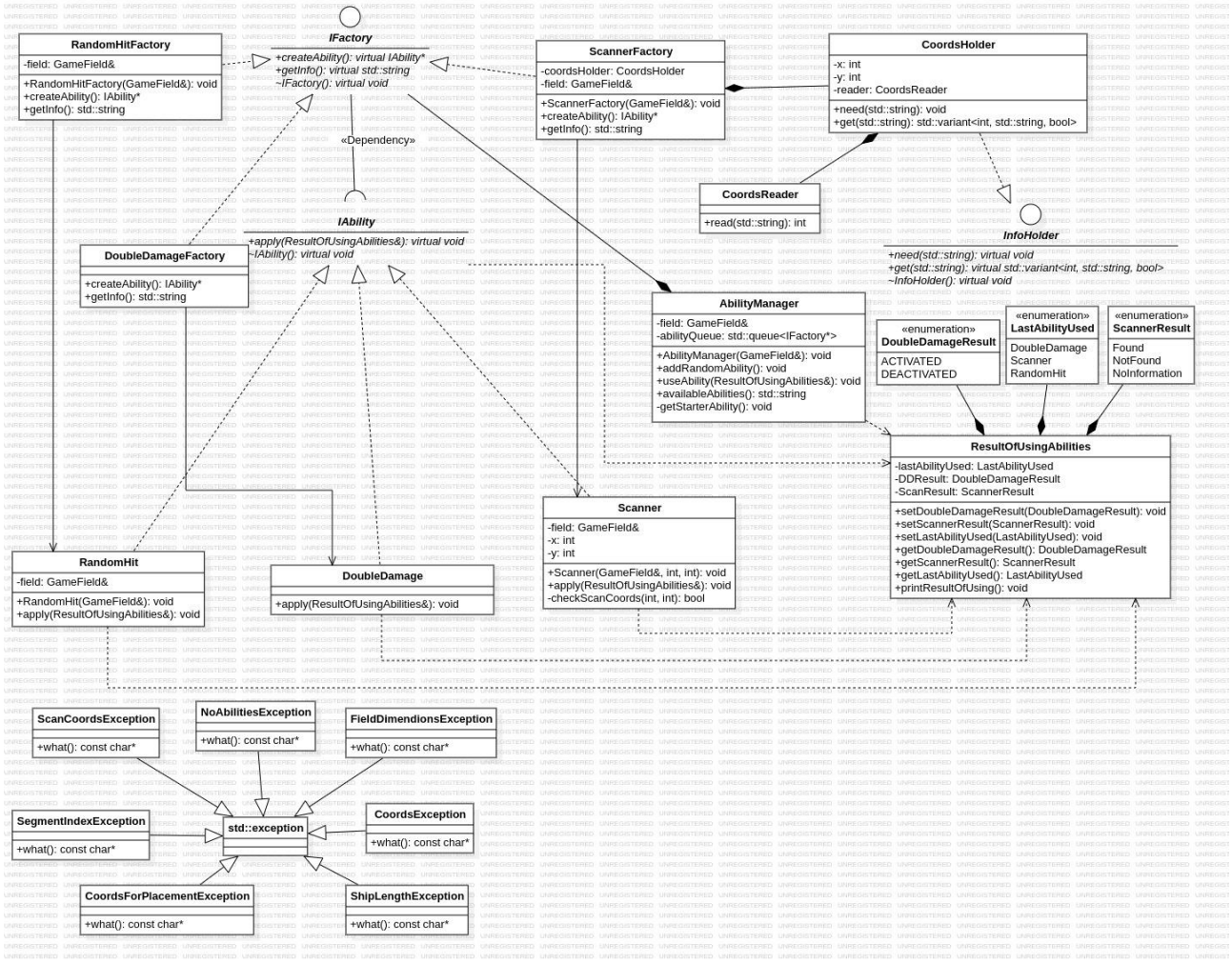
- *int read(std::string name);* - запрашивает у пользователя параметр и возвращает его.

Классы-исключения

Была доработана логика исключений и созданы несколько классов-исключений в которых переопределяется метод *what()* для вывода информации об ошибках при отлавливании через *try-catch*.

- *CoordsException()* – исключение, возникающее при подаче некорректных координат.
- *CoordsForPlacementException()* – исключение, возникающее при попытке установить корабль в неподходящем месте.
- *FieldDimendionsException()* – исключение, возникающее при попытке создать поле некорректных размеров.
- *NoAbilitiesException()* – исключение, возникающее при попытке использовать способность, когда их нет.
- *SegmentIndexException()* – исключение, возникающее при подаче некорректного индекса сегмента корабля.
- *ShipLengthException()* – исключение, возникающее при попытке создать корабль некорректной длины
- *ScanCoordsException()* – исключение, возникающее при выходе поля сканирования за рамки игрового поля.

UML – диаграмма реализованных классов



Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

Входные данные	Выходные данные	Комментарии
std::vector<Length> len(2, Length::Large); len.emplace_back(Length::Small); ShipManager man(3, len); GameField field(10,10,man); AbilityManager abilities(field); ResultOfUsingAbilities res; field.placeShip(man.getShips()[0], Orientation::Horizontal, 0, 5); field.placeShip(man.getShips()[1], Orientation::Horizontal, 2, 3); field.placeShip(man.getShips()[2], Orientation::Horizontal, 0, 0); field.displayField(false); CoordsHolder coordsHolder; std::cout << abilities.availableAbilities() << std::endl; abilities.useAbility(res); if (res.getLastAbilityUsed() == LastAbilityUsed::DoubleDamage) { coordsHolder.need("X"); coordsHolder.need("Y"); auto x_value = coordsHolder.get("X"); auto y_value = coordsHolder.get("Y"); int x, y; if (std::holds_alternative<int>(x_value)) { x = std::get<int>(x_value); } if (std::holds_alternative<int>(y_value)) { y = std::get<int>(y_value); } if (field.attackCell(x, y, res) == true) abilities.addRandomAbility(); } res.printResultOfUsing(); field.displayField(false); std::cout << abilities.availableAbilities() << std::endl; abilities.useAbility(res); if (res.getLastAbilityUsed() == LastAbilityUsed::DoubleDamage) { coordsHolder.need("X"); coordsHolder.need("Y"); auto x_value = coordsHolder.get("X");	[2] [■] [2] [2] [2] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [2] [2] [2] [■] ---------- Доступные способности: [Double Damage], [Scanner], [Random Hit] Введите значение X: 0 Введите значение Y: 0 [Активировано]: Double Damage [X] [■] [2] [2] [2] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [2] [2] [2] [■] ---------- Доступные способности: [Scanner], [Random Hit], [Double Damage] Введите значение X: 2 Введите значение Y: 3 [Активировано]: Scanner [Результат]: Корабль обнаружен [X] [■] [2] [2] [2] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [■] [2] [2] [2] [■]	Корректно

```
        auto y_value = coordsHolder.get("Y");
        int x, y;

        if (std::holds_alternative<int>(x_value)) {
            x = std::get<int>(x_value);
        }
        if (std::holds_alternative<int>(y_value)) {
            y = std::get<int>(y_value);
        }
        if (field.attackCell(x, y, res) == true) abilities.addRandomAbility();
    }
    res.printResultOfUsing();
    field.displayField(false);

    std::cout << abilities.availableAbilities() << std::endl;

    abilities.useAbility(res);
    if (res.getLastAbilityUsed() == LastAbilityUsed::DoubleDamage) {
        coordsHolder.need("X");
        coordsHolder.need("Y");

        auto x_value = coordsHolder.get("X");
        auto y_value = coordsHolder.get("Y");
        int x, y;

        if (std::holds_alternative<int>(x_value)) {
            x = std::get<int>(x_value);
        }
        if (std::holds_alternative<int>(y_value)) {
            y = std::get<int>(y_value);
        }
        if (field.attackCell(x, y, res) == true) abilities.addRandomAbility();
    }
    res.printResultOfUsing();
    field.displayField(false);

    std::cout << abilities.availableAbilities() << std::endl;

    try {
        abilities.useAbility(res);
        if (res.getLastAbilityUsed() == LastAbilityUsed::DoubleDamage) {
            coordsHolder.need("X");
            coordsHolder.need("Y");

            auto x_value = coordsHolder.get("X");
            auto y_value = coordsHolder.get("Y");
            int x, y;

            if (std::holds_alternative<int>(x_value)) {
                x = std::get<int>(x_value);
```

```
---------------------------------------------
-------

Доступные способности: [ Random Hit ], [ Double Damage ]
[ Активировано ]: Random Hit
[X] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [2] [1] [2] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[2] [2] [2] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]
[■] [■] [■] [■] [■] [■] [■] [■] [■] [■]

---------------------------------------------
-------

Доступные способности: [ Double Damage ]
Введите значение X: 2
Введите значение Y: 3
[ Активировано ]: Double Damage
[ ERROR ]: Нет доступных способностей.
```

```
        }
        if (std::holds_alternative<int>(y_value)) {
            y = std::get<int>(y_value);
        }
        if (field.attackCell(x, y, res) == true) abilities.addRandomAbility();
    }
    res.printResultOfUsing();
  }catch(NoAbilitiesException& e) {
    std::cout << e.what() << std::endl;
  }

    try {
    abilities.useAbility(res);
    if (res.getLastAbilityUsed() == LastAbilityUsed::DoubleDamage) {
        coordsHolder.need("X");
        coordsHolder.need("Y");

        auto x_value = coordsHolder.get("X");
        auto y_value = coordsHolder.get("Y");
        int x, y;

        if (std::holds_alternative<int>(x_value)) {
            x = std::get<int>(x_value);
        }
        if (std::holds_alternative<int>(y_value)) {
            y = std::get<int>(y_value);
        }
        if (field.attackCell(x, y, res) == true) abilities.addRandomAbility();
    }
    res.printResultOfUsing();
  }catch(NoAbilitiesException& e) {
    std::cout << e.what() << std::endl;
  }
  return 0;
}
```

Корректно активированы все 3 способности. Корректно выброшено исключение об отсутствии доступных способностей. Корректно выдана и использована дополнительная способность.

Вывод:

В процессе выполнения лабораторной работы был создан интерфейс способностей. Также был разработан менеджер способностей, который позволяет выдавать, перемешивать и активировать способности. Были добавлены исключения для корректной обработки ошибок.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: AbilityManager.cpp

```
#include "AbilityManager.hpp"

AbilityManager::AbilityManager(GameField& field) : field(field) {
    getStarterAbility();
}

void AbilityManager::getStarterAbility(){
            std::vector<IFactory*>    abilitiesFactory    =    {new
DoubleDamageFactory(),       new       RandomHitFactory(field),       new
ScannerFactory(field)};
    std::random_device rd;
    std::default_random_engine rng(rd());
     std::shuffle(abilitiesFactory.begin(),  abilitiesFactory.end(),
rng);

    for (IFactory* abilityFactory : abilitiesFactory) {
        abilityQueue.push(abilityFactory);
    }
}

void AbilityManager::addRandomAbility(){
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> randomAbilitygen(0,2);
    int randomAbility = randomAbilitygen(gen);

    switch(randomAbility) {
        case 0:
            abilityQueue.push(new DoubleDamageFactory());
            break;
        case 1:
            abilityQueue.push(new ScannerFactory(field));
            break;
        case 2:
            abilityQueue.push(new RandomHitFactory(field));
            break;
    }
}

void AbilityManager::useAbility(ResultOfUsingAbilities& results){
    if (abilityQueue.empty()) {
        throw NoAbilitiesException();
    }
    IAbility* ability = abilityQueue.front()->createAbility();
    ability->apply(results);
    abilityQueue.pop();
    delete ability;
}

std::string AbilityManager::availableAbilities() {
```

```
    if (abilityQueue.empty()) {
        return "[ Нет доступных способностей ]";
    } else {
        std::string abilities;
        std::vector<std::string> abilitiesInfo;
        std::queue<IFactory*> tempQueue = abilityQueue;

        while (!tempQueue.empty()) {
            abilitiesInfo.push_back(tempQueue.front()->getInfo());
            tempQueue.pop();
        }

        for (const auto& info : abilitiesInfo) {
            if (!abilities.empty()) {
                abilities += ", ";
            }
            abilities += info;
        }

        return "Доступные способности: " + abilities;
    }
}
```

Название файла: AbilityManager.hpp

```
#ifndef ABILITYMANAGER_H
#define ABILITYMANAGER_H

#include <queue>
#include <vector>
#include <random>
#include <algorithm>

#include "IAbility.hpp"
#include "../Factories/IFactory.hpp"
#include "DoubleDamageAbility.hpp"
#include "ScannerAbility.hpp"
#include "RandomHitAbility.hpp"

#include "../Factories/DoubleDamageFactory.hpp"
#include "../Factories/RandomHitFactory.hpp"
#include "../Factories/ScannerFactory.hpp"

#include "../GameField.hpp"
#include "../Exceptions/NoAbilitiesException.hpp"

#include "ResultOfUsingAbilities.hpp"
```

```
class GameField;

class AbilityManager {
public:
    AbilityManager(GameField& field);
    void addRandomAbility();
    void useAbility(ResultOfUsingAbilities& results);
    std::string availableAbilities();

private:
    GameField& field;
    std::queue<IFactory*> abilityQueue;

    void getStarterAbility();
};

#endif ABILITYMANAGER_H
```

Название файла: DoubleDamageAbility.cpp

```
#include "DoubleDamageAbility.hpp"

void DoubleDamage::apply(ResultOfUsingAbilities& results) {
    results.setDoubleDamageResult(DoubleDamageResult::ACTIVATED);
    results.setLastAbilityUsed(LastAbilityUsed::DoubleDamage);
}
```

Название файла: DoubleDamageAbility.hpp

```
#ifndef DOUBLEDAMAGE_H
#define DOUBLEDAMAGE_H

#include "IAbility.hpp"
#include "../GameField.hpp"

class DoubleDamage : public IAbility {
public:
    void apply(ResultOfUsingAbilities& results);
};

#endif DOUBLEDAMAGE_H
```

Название файла: IAbility.hpp

```
#ifndef ABILITYINTERFACE_H
#define ABILITYINTERFACE_H
```

```
#include <vector>
#include <unordered_map>
#include <string>

#include "ResultOfUsingAbilities.hpp"

class IAbility {
public:
    virtual void apply(ResultOfUsingAbilities& results) = 0;
    virtual ~IAbility() = default;
};

#endif ABILITYINTERFACE_H
```

Название файла: RandomHitAbility.cpp

```
#include "RandomHitAbility.hpp"

RandomHit::RandomHit(GameField& field) : field(field) {}

void RandomHit::apply(ResultOfUsingAbilities& results) {
    int shipIndices = field.getManager().getShips().size();
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> randomShip(0,shipIndices-1);
    int randomShipIndex = randomShip(gen);
                                        std::uniform_int_distribution<>
randomSegment(0,static_cast<int>(field.getManager().getShips()[randomShip
Index].getLength()) - 1);
    int randomSegmentIndex = randomSegment(gen);
                                                                  while
(field.getManager().getShips()[randomShipIndex].getSegmentState(randomSeg
mentIndex) != SegmentState::Intact &&
                field.getManager().getShips()[randomShipIndex].getSegmen
tState(randomSegmentIndex) != SegmentState::Damaged) {
            randomShipIndex = randomShip(gen);
            randomSegmentIndex = randomSegment(gen);
    }

     field.getManager().getShips()[randomShipIndex].hitSegment(rando
mSegmentIndex);
    results.setLastAbilityUsed(LastAbilityUsed::RandomHit);
}
```

Название файла: RandomHitAbility.hpp

```
#ifndef RANDOMHIT_H
#define RANDOMHIT_H

#include "IAbility.hpp"
#include "../GameField.hpp"
#include <random>
#include <cstdlib>

class RandomHit : public IAbility {
public:
    RandomHit(GameField& field);
    void apply(ResultOfUsingAbilities& results);
```

```
private:
    GameField& field;
};

#endif
```

Название файла: ResultOfUsingAbilities.cpp

```
#include "ResultOfUsingAbilities.hpp"

void
ResultOfUsingAbilities::setDoubleDamageResult(DoubleDamageResult  result)
{
    DDResult = result;
}

void  ResultOfUsingAbilities::setScannerResult(ScannerResult  result)
{
    ScanResult = result;
}

void ResultOfUsingAbilities::setLastAbilityUsed(LastAbilityUsed last)
{
    lastAbilityUsed = last;
}

DoubleDamageResult   ResultOfUsingAbilities::getDoubleDamageResult()
const {
    return DDResult;
}

ScannerResult ResultOfUsingAbilities::getScannerResult() const {
    return ScanResult;
}

LastAbilityUsed ResultOfUsingAbilities::getLastAbilityUsed() const {
    return lastAbilityUsed;
}

void ResultOfUsingAbilities::printResultOfUsing() {
    switch(lastAbilityUsed) {
        case LastAbilityUsed::DoubleDamage:
           std::cout << "[ Активировано ]: Double Damage" << std::endl;
            break;
        case LastAbilityUsed::Scanner:
            std::cout << "[ Активировано ]: Scanner" << std::endl;
              if (ScanResult == ScannerResult::Found) std::cout <<
"[ Результат ]: Корабль обнаружен" << std::endl;
             if (ScanResult == ScannerResult::NotFound) std::cout <<
"[ Результат ]: Корабль не найден" << std::endl;
            break;
        case LastAbilityUsed::RandomHit:
           std::cout << "[ Активировано ]: Random Hit" << std::endl;
            break;
    }
}
```

Название файла: ResultOfUsingAbilities.hpp

```
#ifndef RESULT_OF_USING_ABILITIES_H
#define RESULT_OF_USING_ABILITIES_H

#include <iostream>

enum class DoubleDamageResult {
    ACTIVATED,
    DEACTIVATED
};

enum class ScannerResult {
    Found,
    NotFound,
    NoInformation
};

enum class LastAbilityUsed {
    DoubleDamage,
    Scanner,
    RandomHit
};

class ResultOfUsingAbilities {
public:
    void setDoubleDamageResult(DoubleDamageResult result);
    void setScannerResult(ScannerResult result);
    void setLastAbilityUsed(LastAbilityUsed last);
    DoubleDamageResult getDoubleDamageResult() const;
    ScannerResult getScannerResult() const;
    LastAbilityUsed getLastAbilityUsed() const;
    void printResultOfUsing();

private:
    LastAbilityUsed lastAbilityUsed;
    DoubleDamageResult DDResult;
    ScannerResult ScanResult;
};

#endif RESULT_OF_USING_ABILITIES_H
```

Название файла: ScannerAbility.cpp

```
#include "ScannerAbility.hpp"

Scanner::Scanner(GameField& field, int x, int y) : field(field), x(x),
y(y) {}

void Scanner::checkScanCoords(int y, int x) {
     if (y < 0 || y >= field.getHeight() || x < 0 || x >=
field.getWidth()) {
         throw ScanCoordsException();
    }
}

void Scanner::apply(ResultOfUsingAbilities& results) {
    for (int i = y; i < y + 2; i++) {
```

```
            for (int j = x; j < x + 2; j++) {
                checkScanCoords(i, j);
               if (field.getCell(j, i).getCellState() == CellState::Ship
&& field.getCell(j, i).getShipPtr()->isDestroyed() == false) {
                    results.setScannerResult(ScannerResult::Found);
                   results.setLastAbilityUsed(LastAbilityUsed::Scanner);
                    return;
                }
            }
        }
        results.setScannerResult(ScannerResult::NotFound);
        results.setLastAbilityUsed(LastAbilityUsed::Scanner);
    }
     }
                abilities += info;
            }

            return "Доступные способности: " + abilities;
        }
    }
```

Название файла: ScannerAbility.hpp

```
#ifndef SCANNER_H
#define SCANNER_H

#include "IAbility.hpp"
#include "../GameField.hpp"
#include "../Exceptions/ScanCoordsException.hpp"

class Scanner : public IAbility {
public:
    Scanner(GameField& field, int x, int y);
    void apply(ResultOfUsingAbilities& results);

private:
    void checkScanCoords(int y, int x);
    GameField& field;
    int x, y;
};

#endif SCANNER_H
```

Название файла: DoubleDamageFactory.cpp

```
#include "DoubleDamageFactory.hpp"

IAbility* DoubleDamageFactory::createAbility() {
    return new DoubleDamage();
}

std::string DoubleDamageFactory::getInfo() const {
    return "[ Double Damage ]";
}
```

Название файла: DoubleDamageFactory.hpp

```
#ifndef DOUBLE_DAMAGE_FACTORY_H
```

```
#define DOUBLE_DAMAGE_FACTORY_H

#include "IFactory.hpp"
#include "../Abilities/DoubleDamageAbility.hpp"
#include "../CoordsHolder.hpp"

class DoubleDamageFactory : public IFactory {
public:
    IAbility* createAbility();
    std::string getInfo() const;
};

#endif DOUBLE_DAMAGE_FACTORY_H
```

Название файла: IFactory.hpp

```
#ifndef IFACTORY_H
#define IFACTORY_H

#include <iostream>
#include "../GameField.hpp"
#include "../Abilities/IAbility.hpp"
#include "../InfoHolder.hpp"

class IFactory {
public:
    virtual IAbility* createAbility() = 0;
    virtual std::string getInfo() const = 0;
    virtual ~IFactory() = default;
};

#endif IFACTORY_H
```

Название файла: RandomHitFactory.cpp

```
#include "RandomHitFactory.hpp"

RandomHitFactory::RandomHitFactory(GameField& field) : field(field)
{}

IAbility* RandomHitFactory::createAbility() {
    return new RandomHit(field);
}

std::string RandomHitFactory::getInfo() const {
    return "[ Random Hit ]";
}
```

Название файла: RandomHitFactory.hpp

```
#ifndef RANDOM_HIT_FACTORY_H
#define RANDOM_HIT_FACTORY_H

#include "IFactory.hpp"
#include "../Abilities/RandomHitAbility.hpp"

class RandomHitFactory : public IFactory {
public:
```

```
    RandomHitFactory(GameField& field);
    IAbility* createAbility();
    std::string getInfo() const;
private:
    GameField& field;
};

#endif RANDOM_HIT_FACTORY_H
```

Название файла: ScannerFactory.cpp

```
#include "ScannerFactory.hpp"

ScannerFactory::ScannerFactory(GameField& field) : field(field) {}

IAbility* ScannerFactory::createAbility() {
    coordsHolder.need("X");
    coordsHolder.need("Y");

    auto x_value = coordsHolder.get("X");
    auto y_value = coordsHolder.get("Y");
    int x, y;

    if (std::holds_alternative<int>(x_value)) {
        x = std::get<int>(x_value);
    }
    if (std::holds_alternative<int>(y_value)) {
        y = std::get<int>(y_value);
    }
    return new Scanner(field, x, y);
}

std::string ScannerFactory::getInfo() const {
    return "[ Scanner ]";
}
```

Название файла: ScannerFactory.hpp

```
#ifndef SCANNER_FACTORY_H
#define SCANNER_FACTORY_H

#include "IFactory.hpp"
#include "../Abilities/ScannerAbility.hpp"
#include "../CoordsHolder.hpp"

class ScannerFactory : public IFactory {
public:
    ScannerFactory(GameField& field);
    IAbility* createAbility();
    std::string getInfo() const;
private:
    CoordsHolder coordsHolder;
    GameField& field;
};

#endif SCANNER_FACTORY_H
```

Название файла: CoordsHolder.cpp

```
#include "CoordsHolder.hpp"

void CoordsHolder::need(std::string name) {
    if (name == "X") x = reader.read(name);
    if (name == "Y") y = reader.read(name);
}

std::variant<int, std::string, bool> CoordsHolder::get(std::string
name) {
    if (name == "X") return x;
    if (name == "Y") return y;
    return {};
}
```

Название файла: CoordsHolder.hpp

```
#ifndef COORDS_HOLDER_H
#define COORDS_HOLDER_H

#include "InfoHolder.hpp"
#include "CoordsReader.hpp"

class CoordsHolder : public InfoHolder {
public:
    void need(std::string name);
    std::variant<int, std::string, bool> get(std::string name);

private:
    int x = -1, y = -1;
    CoordsReader reader;
};

#endif COORDS_HOLDER_H
```

Название файла: CoordsReader.cpp

```
#include "CoordsReader.hpp"

int CoordsReader::read(std::string name) {
    int parameter;
    std::cout << "Введите значение " + name + ": ";
    std::cin >> parameter;
    return parameter;
}
 }
```

Название файла: CoordsReader.hpp

```
#ifndef COORDS_READER_H
#define COORDS_READER_H

#include <string>
#include <iostream>

class CoordsReader {
public:
```

```
    int read(std::string name);
};

#endif COORDS_READER_H
```

Название файла: InfoHolder.hpp

```
#ifndef INFO_HOLDER_H
#define INFO_HOLDER_H

#include <variant>
#include <string>

class InfoHolder {
public:
    virtual void need(std::string name) = 0;
    virtual std::variant<int, std::string, bool> get(std::string name)
= 0;
    ~InfoHolder() = default;
};

#endif INFO_HOLDER_H
```

Название файла: CoordsException.hpp

```
#ifndef COORDS_EXCEPTION_H
#define COORDS_EXCEPTION_H

#include <exception>

class CoordsException : public std::exception {
public:
    const char* what() const noexcept {
        return "[ ERROR ]: Некорректные координаты.";
    }
};

#endif COORDS_EXCEPTION_H
```

Название файла: CoordsForPlacementException.hpp

```
#ifndef COORDS_FOR_PLACEMENT_EXCEPTION_H
#define COORDS_FOR_PLACEMENT_EXCEPTION_H

#include <exception>

class CoordsForPlacementException : public std::exception {
public:
    const char* what() const noexcept {
         return "[ ERROR ]: Некорректные координаты для размещения
корабля.";
    }
};

#endif COORDS_FOR_PLACEMENT_EXCEPTION_H
```

Название файла: FieldDimendionsException.hpp

```
#ifndef COORDS_DIMENDIONS_EXCEPTION_H
#define COORDS_DIMENDIONS_EXCEPTION_H

#include <exception>

class FieldDimendionsException : public std::exception {
public:
    const char* what() const noexcept {
        return "[ ERROR ]: Некорректные размеры поля.";
    }
};

#endif COORDS_DIMENDIONS_EXCEPTION_H
```

Название файла: NoAbilitiesException.hpp

```
#ifndef NO_ABILITIES_EXCEPTION_H
#define NO_ABILITIES_EXCEPTION_H

#include <exception>

class NoAbilitiesException : public std::exception {
public:
    const char* what() const noexcept {
        return "[ ERROR ]: Нет доступных способностей.";
    }
};

#endif NO_ABILITIES_EXCEPTION_H
```

Название файла: ScanCoordsException.hpp

```
#ifndef SCAN_COORDS_EXCEPTION_H
#define SCAN_COORDS_EXCEPTION_H

#include <exception>

class ScanCoordsException : public std::exception {
public:
    const char* what() const noexcept {
         return "[ ERROR ]: Область сканирования выходит за рамки
поля.";
    }
};

#endif SCAN_COORDS_EXCEPTION_H
```

Название файла: SegmentIndexException.hpp

```
#ifndef SEGMENT_INDEX_EXCEPTION_H
#define SEGMENT_INDEX_EXCEPTION_H

#include <exception>

class SegmentIndexException : public std::exception {
public:
    const char* what() const noexcept {
        return "[ ERROR ]: Некорректный индекс сегмента корабля.";
```

```
    }
};

#endif SEGMENT_INDEX_EXCEPTION_H
```

Название файла: ShipLengthException.hpp

```
#ifndef SHIP_LENGTH_EXCEPTION_H
#define SHIP_LENGTH_EXCEPTION_H

#include <exception>

class ShipLengthException : public std::exception {
public:
    const char* what() const noexcept {
        return "[ ERROR ]: Некорректная длина корабля.";
    }
};

#endif SHIP_LENGTH_EXCEPTION_H
```